

KSTE & The Friedman Sieve: System and Implementation

Paper IV — Engineering the Friedman Stack on Snapdragon V69

KnackAU, Claude (Anthropic), Gemini (Google DeepMind)

Shannon-Prime Project · 2026-05-19

Abstract

This paper specifies the system that realizes the Friedman Stack of Paper III on existing Shannon-Prime infrastructure. The central contribution is the *Knight-Spinor Tree Encoder* (KSTE) — the exact, computable function $\Phi : \mathbb{R}^{128} \rightarrow \mathcal{T}_{60,3}$ that maps a continuous Key vector to a 60-node, 3-label rooted tree, fitting inside the existing 63-byte packed block. We give the full encoder kernel in pseudocode and C, the homeomorphic-embedding decision kernel ($O(|T_Q| \cdot |T_K|)$ on CPU, fully vectorisable on Hexagon HVX via predicate-register operations), and the Friedman-sieve integration into the `sp_hex_sqfree_cache_t` write path. We then describe the test plan, the parity gates, and the experimental schedule for the perplexity / eviction-rate benchmarks that decide whether the framework ships as default or remains an opt-in path.

1. System Position

Paper III’s theory rests on KSTE. If the encoder cannot be implemented cheaply, the theory is academic; if it can, the Friedman Sieve becomes a drop-in replacement for variance-ranked top-K eviction with mathematical guarantees attached.

This paper is the engineering specification. It assumes Papers I (theory) and II (system) and Paper III (Friedman theory) as prerequisites.

2. Block Format

The existing 63-byte Spinor block (Paper II §9.1) is reused without modification:

Bytes 0– 27 : 14 × fp16 anchor coefficients (Knight skeleton)
Bytes 28– 58 : 60 lanes × (3-bit magnitude + 1-bit phase) = 31 bytes
Bytes 59– 62 : 4-byte amax (block-level scaling)

The KSTE encoder constructs the tree from this exact byte layout. No additional storage is required. The tree representation is *derived*, not stored: each block carries enough information to reconstruct $\Phi(K)$ on demand in $O(60)$ operations.

For the *evicted* trees (the ones we keep in cache as eviction comparators), we add a separate 60-byte packed-tree representation:

```
typedef struct {  
    uint8_t labels [15];    /* 60 nodes × 2 bits */
```

```

    uint8_t parents[45];    /* 60 nodes × 6 bits */
} sp_kste_tree;           /* 60 bytes packed */

```

Total per-cache-slot footprint: 63 (existing) + 60 (tree) = **123 bytes** per K slot.

3. The KSTE Encoder

3.1 High-level pseudocode

```

function KSTE(block: sp_ok_q8_block_t) -> sp_kste_tree:
    # 1. Extract anchors and residuals from existing block
    anchors[0..13] = fp16_decode(block.bytes[0..27])
    (mag[0..59], phase[0..59]) = unpack_residuals(block.bytes[28..58])
    amax = fp32_decode(block.bytes[59..62])

    # 2. Compute order-invariant signature
    rank_a[k] = rank-of(|anchors[k]|) for k in 0..13
    sign_a[k] = sign(anchors[k])      for k in 0..13

    # 3. Build tree
    T = new_tree()
    root = T.add_root(label = ROOT)

    # 3a. Add anchor children, ordered by rank
    for k in argsort(rank_a, descending):
        if |anchors[k]| < tau_A * amax:
            break          # below threshold; stop
        T.add_child(root, label = A)

    # 3b. Attach residuals to dominating anchors
    for j in 0..59:
        if mag[j] == 0:
            continue
        # Phase determines label
        label = B if phase[j] == 0 else C
        # Find dominating anchor
        target_rank = rank-of-magnitude(mag[j], amax)
        k_star = first k such that rank_a[k] > target_rank * alpha
        parent = (T.children(root))[k_star] if k_star exists else root
        # Magnitude becomes depth: chain of length mag[j]
        current = parent
        for d in 0..(mag[j] - 1):
            current = T.add_child(current, label = label)

    # 4. Budget enforcement
    while T.node_count > 60:
        T.remove_weakest_leaf() # leaf with lowest |residual|

```

```
return T.pack() # → sp_kste_tree (60 bytes)
```

3.2 C reference implementation

```
/* sp_kste.h */
#define SP_KSTE_MAX_NODES    60
#define SP_KSTE_TAU_A        0.05f
#define SP_KSTE_ALPHA        0.7f

typedef enum { SP_KSTE_ROOT = 0, SP_KSTE_A = 1, SP_KSTE_B = 2, SP_KSTE_C = 3 } sp_kste_

typedef struct {
    uint8_t labels [15]; /* packed 2-bit labels */
    uint8_t parents[45]; /* packed 6-bit parent indices */
    uint8_t node_count;
    uint8_t _pad[3];
} sp_kste_tree; /* sizeof = 64 bytes (60 packed + 4 metadata) */

/* sp_kste.c */
void sp_kste_encode(const sp_ok_q8_block_t *block, sp_kste_tree *out);
int  sp_kste_embed (const sp_kste_tree *Q, const sp_kste_tree *K); /* 1 iff Q ⊆ K */
```

The encoder has no floating-point divisions; all magnitude comparisons are integer ranks. The single fp16→fp32 step on the 14 anchors is the only non-integer work and is bounded to $O(14)$ per block.

3.3 Determinism and stability

The encoder is **deterministic and order-invariant** by construction. Two Key vectors with the same Knight-Skeleton-Spinor signature produce the *bit-identical* tree. The encoder is stable under any rescaling of K that preserves the rank pattern and phase pattern — exactly the order-invariance of Paper III §4.1.

This means the encoder is also stable under Frobenius shimming (Paper I §3.2): the π^k scale factor preserves ranks and signs, so $\Phi(\pi^k K) = \Phi(K)$. The KSTE layer is *invariant* under Frobenius — a property that matters because the engine never has to decide whether to encode before or after the shim.

4. The Homeomorphic-Embedding Kernel

4.1 Decision procedure

The relation $T_Q \preceq T_K$ is decidable in polynomial time. The standard algorithm is the *ordered tree embedding* dynamic program (Kilpeläinen-Mannila, 1995), $O(|T_Q| \cdot |T_K|^2)$ in the worst case, with optimizations bringing typical-case complexity down to $O(|T_Q| \cdot |T_K|)$ when many partial embeddings fail early.

For 60-node trees the worst case is $\approx 216,000$ operations, easily inside one HVX dispatch.

4.2 Packed bitwise form

Both T_Q and T_K are stored as `sp_kste_tree` (60-byte packed). The embedding test in vectorised form:

```
function sp_kste_embed_hvx(Q: tree, K: tree) -> {0,1}:
    # Compare label masks: for each Q label, find candidate K nodes
    # with matching label using vcmpeq predicate
    cand_A = vcmpeq(K.labels, A_mask)    # HVX 64-byte vector
    cand_B = vcmpeq(K.labels, B_mask)
    cand_C = vcmpeq(K.labels, C_mask)

    # For each Q node, mask candidate K nodes by label
    # then check parent-ancestor relation via parent-pointer transitive closure
    K_anc = transitive_closure(K.parents)  # precomputed once per K

    # Run the DP greedily under vector-predicate guard
    # ... see §4.3 for the full kernel
```

The transitive-closure of parent pointers is computed once per K and cached alongside the packed tree (an additional $60 \times 60 / 8 = 450$ bytes, or rounded to a 512-byte block for alignment). The total cache slot becomes $123 + 512 = \mathbf{635 \text{ bytes per K slot}}$ for the embedding-accelerated form.

If memory is tight, the closure can be recomputed at test time at $O(60^2/8) = 450$ ops per K, well inside HVX budget.

4.3 Hexagon kernel sketch

```
/* sp_hex_kste_embed.idl */
int sp_hex_kste_embed(
    const uint8_t *Q_packed,    /* 60-byte tree */
    const uint8_t *K_packed,    /* 60-byte tree */
    uint8_t *result              /* 0 or 1 */
);

/* sp_hex_kste_embed_imp.c - DSP-side */
int sp_hex_kste_embed(...) {
    HVX_Vector vQ_labels = *(HVX_Vector *)Q_packed;
    HVX_Vector vK_labels = *(HVX_Vector *)K_packed;

    /* For each label class, build a vector predicate of K-candidates */
    HVX_VectorPred pA = Q6_Q_vcmp_eq_VbVb(vK_labels, vAconst);
    HVX_VectorPred pB = Q6_Q_vcmp_eq_VbVb(vK_labels, vBconst);
    HVX_VectorPred pC = Q6_Q_vcmp_eq_VbVb(vK_labels, vCconst);

    /* Walk Q in BFS order, maintaining the current "must descend from" frontier */
    /* Frontier represented as a 60-bit mask (one bit per K node) */
    uint64_t frontier = 1ULL; /* root */
```

```

for (int q = 1; q < Q.node_count; ++q) {
    uint8_t q_lbl = unpack_label(Q.labels, q);
    uint8_t q_par = unpack_parent(Q.parents, q);
    uint64_t cand = (q_lbl == A) ? pA_mask : (q_lbl == B) ? pB_mask : pC_mask;
    /* Restrict cand to descendants of frontier */
    uint64_t reach = expand_descendants(frontier, K_anc);
    uint64_t valid = cand & reach;
    if (valid == 0) return 0;
    /* Pick lowest-rank candidate (smallest index in K's BFS order) */
    int chosen = __builtin_ctzll(valid);
    frontier = (1ULL << chosen);
}
return 1;
}

```

This is a greedy embedding; full correctness requires backtracking when the greedy choice fails. The full algorithm uses a 60-element stack of partial assignments and runs in $\leq 60 \times 60 = 3600$ steps worst-case. The kernel fits in VTCM with room to spare.

5. Sieve Integration

5.1 Cache structure

```

typedef struct {
    sp_ok_q8_block_t k_block;      /* 63 bytes – existing */
    sp_kste_tree k_tree;          /* 60 bytes – new */
    uint64_t k_anc_mask;          /* 8 bytes – transitive closure of parent pointers */
    /* total: 131 bytes per slot, 64-byte aligned to 192 bytes */
} sp_friedman_slot_t;

```

5.2 Write path

```

int sp_friedman_cache_write(
    sp_friedman_cache_t *cache,
    const float *K_vec,          /* incoming Key */
    int layer, int head, int pos
) {
    /* 1. Existing pipeline: pack K into 63-byte Spinor block */
    sp_ok_q8_block_t new_block;
    sp_hex_compress_f32(K_vec, head_dim, &new_block);

    /* 2. KSTE: derive tree from block */
    sp_kste_tree new_tree;
    sp_kste_encode(&new_block, &new_tree);

    /* 3. Sieve test: does new_tree embed into ANY existing tree? */
    for (int t = 0; t < cache->count; ++t) {
        if (sp_kste_embed(&new_tree, &cache->slots[t].k_tree)) {

```

```

        /* Subsumed; evict (i.e., do nothing – token absorbed) */
        cache->eviction_count++;
        return SP_FRIEDMAN_EVICTED;
    }
}

/* 4. Novel; add to cache */
if (cache->count >= cache->capacity) {
    /* Cache full; evict by Knight-Skeleton variance (fallback) */
    sp_friedman_evict_weakest(cache);
}
int slot = cache->count++;
cache->slots[slot].k_block = new_block;
cache->slots[slot].k_tree = new_tree;
cache->slots[slot].k_anc_mask = sp_kste_anc_closure(&new_tree);
return SP_FRIEDMAN_ADMITTED;
}

```

5.3 Read path

Attention scoring proceeds as before — the polynomial-ring kernel reads `k_block` and computes the dot product with the current `Q`. The tree representation is *write-only* from the attention kernel’s perspective: it is consulted only by the sieve.

This decoupling means the sieve is *purely* an admission policy. Attention scoring is unchanged. PPL effect is bounded by the eviction-rate $\times$ per-evicted-token information content.

6. Experimental Schedule

The experiments fall into three tiers. Each tier has an explicit ship/no-ship criterion.

6.1 Tier 1: Encoder sanity

Test	Inputs	Pass criterion
T1.1 — Encoder is deterministic	Same $K \times 1000$ trials	Bit-identical tree across all trials
T1.2 — Encoder is order-invariant	K vs $\pi^k K \times 100$ trials	Bit-identical tree
T1.3 — Encoder is sign- respecting	K vs $-K$	Trees differ only in B $\leftrightarrow$ C label swaps
T1.4 — Budget enforced	$K \sim \mathcal{N}(0, I_{128}) \times$ 1000	$ V(\Phi(K)) \leq 60$ always

Test	Inputs	Pass criterion
T1.5 — Anchor count matches	Top-14 selection	14 ± 2 children of root

All five tests must pass before any sieve testing begins.

6.2 Tier 2: Sieve behaviour

Test	Inputs	Pass criterion
T2.1 — Sieve termination	100k random tokens, encoder fixed	Cache size $\leq \text{antichain}(\mathcal{T}_{60,3})$
T2.2 — Eviction rate	WikiText-103, ctx=2048	$\geq 20\%$ at steady state
T2.3 — Sieve PPL ablation	Gemma3-1B, sieve on/off	$\Delta\text{PPL} \leq 0.5\%$
T2.4 — Refutation procedure	Inject adversarial K	Counterexample found in $\leq 10^4$ ops
T2.5 — Closure-axiom invariant	Cache snapshot, random subsets	$\text{Big} \cap \text{Big} = \text{Big}$ for all pairs

T2.3 is the gating test: if PPL drift exceeds 0.5%, the sieve is not shippable as default. Tier 3 is contingent on T2.3 passing.

6.3 Tier 3: Ultraproduct attention

Test	Inputs	Pass criterion
T3.1 — Principal $\Rightarrow$ Top-1	Toy 16-token cache	UltraAttn = Top-1 exactly
T3.2 — Łoś on toy	Hand-crafted properties	Holds at limit iff U -large set
T3.3 — PPL on LongBench	UltraAttn vs softmax	$\Delta\text{PPL} \leq 3\%$ either way
T3.4 — RULER long-context	UltraAttn at $n \geq 32k$	Match or beat softmax
T3.5 — Wall-time	HVX kernel vs softmax	Within 20% wall-time

Tier 3 is a research experiment. Even a +3% PPL with −50% wall-time would be publishable; the framework’s value is the new attention primitive, not strict PPL.

7. Build and Wire-Up

7.1 New files

```
lib/shannon-prime/  
  sp_kste.h          (encoder + embed API)  
  sp_kste.c          (CPU reference impl)  
  sp_kste_pack.c     (bit-packing helpers)
```

```
shannon-prime-engine/  
  src/sp_friedman_cache.h  
  src/sp_friedman_cache.cpp  
  src/sp_ultraproduct_attn.h  
  src/sp_ultraproduct_attn.cpp  
  src/sp_hex_kste_embed.idl  
  src/sp_hex_kste_embed_imp.c
```

```
tests/  
  test_sp_kste.cpp  
  test_sp_friedman_cache.cpp  
  test_sp_ultraproduct_attn.cpp
```

Approximate LOC budget: encoder 200, embedding 400, sieve integration 500, ultra-product attention 600, tests 1500. **Total ~3200 LOC.**

7.2 CMake hooks

```
option(SP_FRIEDMAN_SIEVE      "Enable Friedman sieve eviction" ON)  
option(SP_KSTE_ENCODER        "Enable KSTE encoder for KV-trees" ON)  
option(SP_ULTRAPRODUCT_ATTN   "Enable ultraproduct attention path" OFF)
```

7.3 CLI flags

```
sp-engine.exe perplexity-sp \  
  --model gemma3-1b.gguf \  
  --frobenius-quant -p 41 -k 8 \  
  --poly-attn --ntt-crt \  
  --friedman-sieve \  
  --kste-tau-A 0.05 --kste-alpha 0.7 \  
  --ultraproduct-attn=principal \  
  --ctx 4096 --chunks 4
```

7.4 Environment variables

Variable	Default	Effect
SP_FRIEDMAN_ENABLE	0	Enable sieve at runtime
SP_FRIEDMAN_CAPACITY	4096	Maximum cache slots after sieve dedup
SP_KSTE_TAU_A	0.05	Anchor inclusion threshold ($\times$ amax)

Variable	Default	Effect
SP_KSTE_ALPHA	0.7	Residual-to-anchor attachment ratio
SP_ULTRAPRODUCT_MODE	none	none / principal / nonprincipal

8. Measurement and Reporting

Each tier produces a structured report:

```
{
  "tier": 2,
  "test_id": "T2.3",
  "config": {
    "model": "gemma3-1b",
    "ctx": 2048,
    "frobenius": "p=41,k=8",
    "sieve": "on"
  },
  "metrics": {
    "ppl_with_sieve": 11.83,
    "ppl_without": 11.83,
    "delta_pct": 0.00,
    "eviction_rate": 0.34,
    "tree_size_mean": 41.2,
    "embed_test_us": 18
  },
  "verdict": "PASS"
}
```

These JSON reports are committed alongside the engine; they form the *audit trail* for the framework’s empirical claims.

9. Known Risks

1. **Encoder discriminative resolution.** The KSTE encoder may not separate close-but-distinct semantic Keys. If T2.3 fails (PPL drift > 0.5%), the encoder is the prime suspect; mitigations include increasing anchor count from 14 to 20, or extending labels to 4-bit (a non-trivial block-format change).
2. **Eviction rate may be too high.** If the sieve evicts > 80% of tokens, novel structure is being lost. Mitigation: tighten the embedding test (require *strict* embedding rather than allowing trivial ones).
3. **Eviction rate may be too low.** If < 5% of tokens are evicted, the sieve is doing no useful work. Mitigation: loosen the embedding test, or add a magnitude-similarity gate.
4. **HVX dispatch density.** The sieve adds one embedding test per write. At 32 layers × 8 heads × 4k tokens, that is 1M tests per inference pass. At 18 μs per

HVX embed (T3.5 budget), that is 18 seconds of compute. Mitigation: batched embedding tests (Strike 16 analog).

5. **Determinism on shipped builds.** The KSTE encoder is deterministic *given the calibration knight-mask*. Builds with different calibrations will produce different trees from the same Keys. Mitigation: ship the knight-mask with the model; treat it as a model artefact.

10. The Choice Operator Implementation

Paper III §11 introduces the axiomatic layer: a choice operator F and the Extended-Domain Reduction axiom. This section specifies the system-side implementation.

10.1 The canonical total order $\prec_F$

For determinism, the choice operator requires a fixed total order on $\mathcal{T}_{60,3}$. We use the *packed lexicographic* order on the 60-byte `sp_kste_tree` representation: compare the two byte arrays component-wise, return the comparison of the first differing byte. The order is trivially computable in $O(60)$ operations, deterministic across all platforms, and stable under the encoder’s invariants — a tree and its bit-equivalent representation compare equal.

```
/* sp_kste.h */
int sp_kste_compare(const sp_kste_tree *a, const sp_kste_tree *b);
/* returns <0 if a <_F b, 0 if equal, >0 if a >_F b */
```

10.2 The selector

$F(A)$ is computed by linear scan with the comparator:

```
/* sp_kste.h */
const sp_kste_tree *sp_kste_select_canonical(
    const sp_kste_tree **candidates,
    int n_candidates
);
/* returns the <_F-minimum tree in the candidate set */
```

Complexity: $O(n \cdot 60)$ for n candidates. At cache size 4096, the worst-case selection is $\approx 245,000$ byte comparisons — well inside a single HVX dispatch and below the embedding-test budget.

10.3 Integration with the sieve

The sieve’s admission test (§5.2) currently returns a boolean. Under the axiomatic layer, the admission decision is augmented:

```
typedef enum {
    SP_FRIEDMAN_EVICTED,
    SP_FRIEDMAN_REPLACED /* admitted as F(A) for some matching class */
} sp_friedman_decision;
```

The REPLACED path is the choice-operator’s behaviour: when a class of cached trees matches the new tree’s structural query, the engine selects $F(A)$ and may replace the entries of A in the cache with the canonical representative. This is a *compaction* operation — the cache shrinks toward canonical witnesses over time, bounded by the antichain count of $\mathcal{T}_{60,3}$ from Theorem 3.2.

10.4 The Extended-Domain Reduction unit test

The reduction axiom is a checkable invariant. For any structural predicate φ and any selected $v = F(A)$, the axiom asserts $\varphi(v) \Rightarrow \varphi^*(v)$, where φ^* is the relativization of φ to the active-window subset of the cache. We implement a small library of standard predicates and a top-level check routine:

```
typedef int (*sp_predicate_t)(const sp_kste_tree *T);

int sp_predicate_anchor_count(const sp_kste_tree *T);
int sp_predicate_label_b_count(const sp_kste_tree *T);
int sp_predicate_max_depth(const sp_kste_tree *T);
/* ... extensible per workload ... */

int sp_extended_reduction_check(
    const sp_kste_tree *v,           /* canonical witness, v = F(A) */
    sp_predicate_t phi,
    const sp_friedman_cache_t *cache_R0 /* active window cache */
);
/* returns 1 iff phi(v) implies phi restricted to R0 */
```

This routine is the engineering realization of the axiom. It runs a finite scan of `cache_R0`, evaluates φ on each entry, and verifies the implication. It is primitive-recursive: no unbounded search, no real-valued arithmetic. See `TEST-SUITE.md` T2.12 for the gating test.

10.5 Performance characteristics

The choice-operator layer adds a measurable, bounded cost per token:

- Canonical comparison: $O(60)$ per pair.
- Selection over candidate class of size k : $O(60k)$.
- Reduction check across $|RO|$: $O(|RO| \cdot \text{cost}(\varphi))$.

At cache size 4096 and typical predicate cost $O(60)$, the per-token CPU overhead is $\leq 250 \mu\text{s}$ — well inside the existing wall-time budget. On Hexagon HVX the comparison is parallelised to one vector instruction per pair, dropping the cost by an order of magnitude. Total axiomatic-layer overhead at production scale: $\leq 5\%$ of attention wall-time.

10.6 Build and CLI

```
option(SP_CHOICE_OPERATOR "Enable axiomatic choice operator + ED Reduction" ON)
```

```

sp-engine.exe perplexity-sp \
  --model gemma3-1b.gguf \
  --frobenius-quant -p 41 -k 8 \
  --poly-attn --ntt-crt \
  --friedman-sieve \
  --choice-operator \
  --reduction-radius 0    # r in Paper III §11.4; 0 = strict consistency
--reduction-radius is the fuzzy-class radius from Paper III §11.4. The default  $r = 0$ 
enforces strict consistency; higher values approach soft-attention behaviour without
ever instantiating a softmax. Per-workload defaults (code:  $r = 0$ ; chat:  $r$  tuned) live
in docs/CHOICE-OPERATOR-RADII.md.

```

11. Conclusion

The Friedman Stack is implementable on existing Shannon-Prime infrastructure with ~ 3500 LOC of new code (including the axiomatic-layer additions of §10) and zero new memory layouts on the cache hot path. The encoder is order-invariant, deterministic, and reuses the existing Knight-Skeleton/Spinor decomposition. The embedding test fits in HVX with the same VTCM footprint as the existing compress/decode pipeline. The sieve is purely an admission policy; the choice operator adds a canonicity primitive that compacts the cache toward $\prec_F$ -minimal witnesses; the Extended-Domain Reduction axiom is unit-testable as a primitive-recursive structural check. Attention scoring remains the polynomial-ring + CRT-NTT kernel of Paper II.

The experiments of §6 (extended by T2.12 and T3.6 in TEST-SUITE.md) decide whether the framework ships. The ship/no-ship gate is T2.3 — a $\leq 0.5\%$ PPL drift with the sieve enabled on Gemma3-1B at ctx=2048. If T2.3 passes, the Friedman Stack becomes the default eviction policy in the next release; if not, it remains an opt-in path while the encoder is iterated.

The next session continues with the implementation roadmap of IMPLEMENTATION-ROADMAP.md and the test specifications of TEST-SUITE.md.

References

1. Kilpeläinen, P. & Mannila, H., *Ordered and Unordered Tree Inclusion*, SIAM J. Comput. 24, 1995.
2. Friedman, H. M., *FOM Embedded Maximal Clique posts*, 2009–2018.
3. Łoś, J., *Quelques remarques, théorèmes et problèmes sur les classes définissables d'algèbres*, 1955.
4. Qualcomm, *Hexagon V69 HVX Intrinsics Reference*, 2023.
5. Shannon-Prime, *Paper III — The Friedman Stack* (theory, this work's companion).
6. Shannon-Prime, *Paper I, II* (Frobenius framework + engine).
7. Anderson, C. A., *Some Emendations on Gödel's Ontological Proof*, 1990.
8. Hilbert, D. & Bernays, P., *Grundlagen der Mathematik II*, 1939 (epsilon-operator).

9. Ackermann, W., *Zur Widerspruchsfreiheit der Zahlentheorie*, Math. Ann. 117, 1940.

This paper is consumed by IMPLEMENTATION-ROADMAP.md. Tests are tracked in TEST-SUITE.md.